

```

#include <iostream>
#include <string>
using namespace std;

struct Node
{
    int digit;
    Node* next;
};

class bigint
{
public:
    Node * head;
    bool negative;

    bigint()
    {
        head = new Node;
        head->digit = 0;
        head->next = NULL;
        negative = false;
    }

    bigint(string s)
    {
        head = NULL;
        negative = false;

        if (s[0] == '-')
        {
            negative = true;
            s = s.substr(1);
        }

        for (int n = 0; n < s.length(); n+=1)
        {
            Node * temp = new Node;
            temp->digit = s[n] - '0';
            temp->next = head;
            head = temp;
        }
    }
}

```

```

bigint(const bigint& other)
{
    negative = other.negative;
    head = NULL;

    Node * curr = other.head;
    Node * tail = NULL;

    while (curr != NULL)
    {
        Node * temp = new Node;
        temp->digit = curr->digit;
        temp->next = NULL;

        if (head == NULL)
        {
            head = temp;
            tail = temp;
        }
        else
        {
            tail->next = temp;
            tail = temp;
        }

        curr = curr->next;
    }
}

string toString()
{
    string s = "";
    Node * curr = head;
    while (curr != NULL)
    {
        s = char(curr->digit + '0') + s;
        curr = curr->next;
    }

    return s;
}

void removeLeadingZeros()
{

```

```

string s = toString();

int n = 0;
while (n < s.length() - 1 && s[n] == '0')
{
    n+=1;
}

s = s.substr(n);

head = NULL;

for (int j = 0; j < s.length(); j+=1)
{
    Node* temp = new Node;
    temp->digit = s[j] - '0';
    temp->next = head;
    head = temp;
}
}

int compareAbs(bigint b)
{
    string a = toString();
    string bb = b.toString();

    if (a.length() > bb.length()) return 1;
    if (a.length() < bb.length()) return -1;

    if (a > bb) return 1;
    if (a < bb) return -1;
    return 0;
}

void print()
{
    string s = toString();
    int count = 0;

    if (negative) cout << "-";
    else cout << " ";

    for (int n = 0; n < s.length(); n+=1)
    {

```

```

        cout << s[n];
        count+=1;

        if (count == 50)
        {
            cout << endl << " ";
            count = 0;
        }
    }
    cout << endl;
}

bigint addAbs(bigint b)
{
    bigint result;
    result.head = NULL;

    Node * p = head;
    Node * q = b.head;
    Node * tail = NULL;

    int carry = 0;

    while (p != NULL || q != NULL || carry != 0)
    {
        int sum = carry;

        if (p != NULL)
        {
            sum += p->digit;
            p = p->next;
        }
        if (q != NULL)
        {
            sum += q->digit;
            q = q->next;
        }

        Node * temp = new Node;
        temp->digit = sum % 10;
        temp->next = NULL;

        if (result.head == NULL)
        {

```

```

        result.head = temp;
        tail = temp;
    }
    else
    {
        tail->next = temp;
        tail = temp;
    }

    carry = sum / 10;
}

return result;
}

bigint subAbs(bigint b)
{
    bigint result;
    result.head = NULL;

    Node * p = head;
    Node * q = b.head;
    Node * tail = NULL;

    int borrow = 0;

    while (p != NULL)
    {
        int diff = p->digit - borrow;

        if (q != NULL)
        {
            diff -= q->digit;
            q = q->next;
        }

        if (diff < 0)
        {
            diff += 10;
            borrow = 1;
        }
        else
        {
            borrow = 0;

```

```

    }

    Node * temp = new Node;
    temp->digit = diff;
    temp->next = NULL;

    if (result.head == NULL)
    {
        result.head = temp;
        tail = temp;
    }
    else
    {
        tail->next = temp;
        tail = temp;
    }

    p = p->next;
}

return result;
}

bigint add(bigint b)
{
    bigint result;

    if (negative == b.negative)
    {
        result = addAbs(b);
        result.negative = negative;
    }
    else
    {
        if (compareAbs(b) >= 0)
        {
            result = subAbs(b);
            result.negative = negative;
        }
        else
        {
            result = b.subAbs(*this);
            result.negative = b.negative;
        }
    }
}

```

```

    }

    result.removeLeadingZeros();
    return result;
}

bigint subtract(bigint b)
{
    b.negative = !b.negative;
    bigint res = add(b);
    res.removeLeadingZeros();
    return res;
}

void multiplyInt(int x)
{
    Node * curr = head;
    int carry = 0;

    while (curr != NULL)
    {
        int prod = curr->digit * x + carry;
        curr->digit = prod % 10;
        carry = prod / 10;

        if (curr->next == NULL)
        {
            while (carry > 0)
            {
                Node* temp = new Node;
                temp->digit = carry % 10;
                temp->next = NULL;
                curr->next = temp;
                curr = temp;
                carry /= 10;
            }
        }

        curr = curr->next;
    }
}

bigint multiply(bigint b)
{

```

```

bigint result("0");

Node * q = b.head;
int pos = 0;

while (q != NULL)
{
    bigint temp = * this;
    temp.multiplyInt(q->digit);

    for (int n = 0; n < pos; n+=1)
    {
        Node * zero = new Node;
        zero->digit = 0;
        zero->next = temp.head;
        temp.head = zero;
    }

    result = result.add(temp);

    pos+=1;
    q = q->next;
}

result.negative = (negative != b.negative);
result.removeLeadingZeros();
return result;
}

void divideInt(int x)
{
    string s = toString();
    string result = "";

    int remainder = 0;

    for (int n = 0; n < s.length(); n+=1)
    {
        int num = remainder * 10 + (s[n] - '0');
        result += char((num / x) + '0');
        remainder = num % x;
    }

    head = NULL;

```

```

    for (int n = 0; n < result.length(); n+=1)
    {
        Node* temp = new Node;
        temp->digit = result[n] - '0';
        temp->next = head;
        head = temp;
    }

    removeLeadingZeros();
}

bigint divide(bigint b)
{
    bigint low("0");
    bigint high = *this;
    bigint one("1");
    bigint mid;

    while (low.compareAbs(high) <= 0)
    {
        mid = low.add(high);
        mid.divideInt(2);

        bigint prod = b.multiply(mid);

        int cmp = prod.compareAbs(*this);

        if (cmp == 0) return mid;
        else if (cmp < 0) low = mid.add(one);
        else high = mid.subtract(one);
    }

    return high;
}

void factorial()
{
    int n = 0;

    string s = toString();
    for (int i = 0; i < s.length(); i+=1)
    {
        n = n * 10 + (s[i] - '0');
    }
}

```

```

    bigint result;
    result = bigint("1");

    for (int i = 2; i <= n; i+=1)
    {
        result.multiplyInt(i);
    }

    *this = result;
}
};

int main()
{
    bigint a("555198537"), b("2000000000"), c("777");
    bigint d = a.multiply(b).add(c);
    d.print();

    bigint x("425");
    x.factorial();
    x.print();

    bigint i("45"), j("11276");
    bigint k = i.subtract(j);
    k.print();

    bigint p("1000000"), q("2");
    bigint r = p.divide(q);
    r.print();
}

```

RESULTS:

[illegible]